Universidade de Fortaleza

CCT

CENTRO DE CIÊNCIAS TECNOLÓGICAS

Resolução de Problemas com Grafos

UNID. 3 Minimal Spanning Tree - MST.

Prof. Me. Ricardo Carubbi (carubbi@unifor.br)

1. Árvore Geradora Mínima - MST

1.1 Introdução

***Dado**: Grafo não direcionado G com pesos positivos nas arestas (conexo).*

***Def.**: Uma **árvore geradora** de G é um subgrafo T que é simultaneamente uma árvore (conexa e acíclico) e geradora (inclui todos os vértices).*

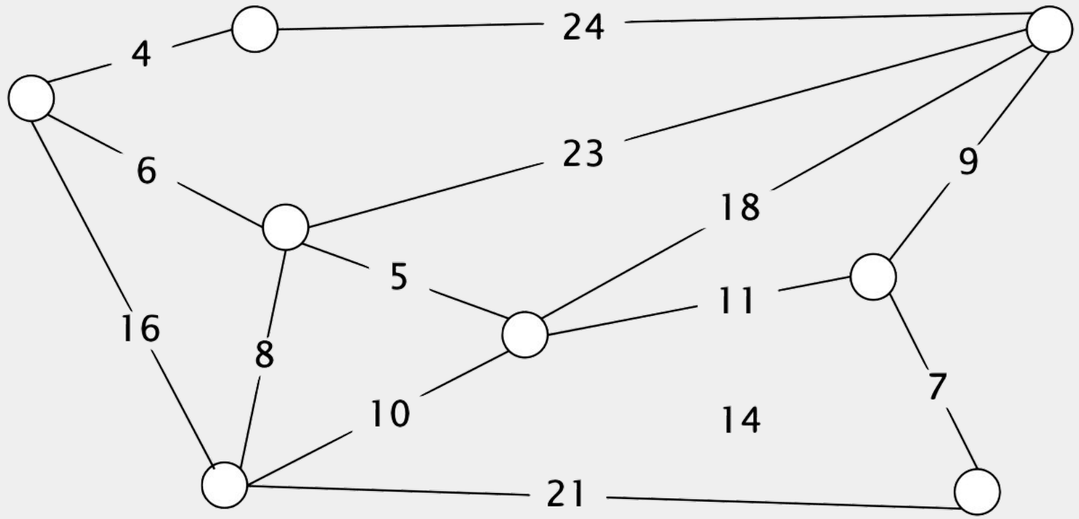
***Objetivo**: Encontrar uma **árvore geradora de peso mínimo**.*

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

1. Árvore Geradora Mínima - MST

1.1 Introdução



Grafo G

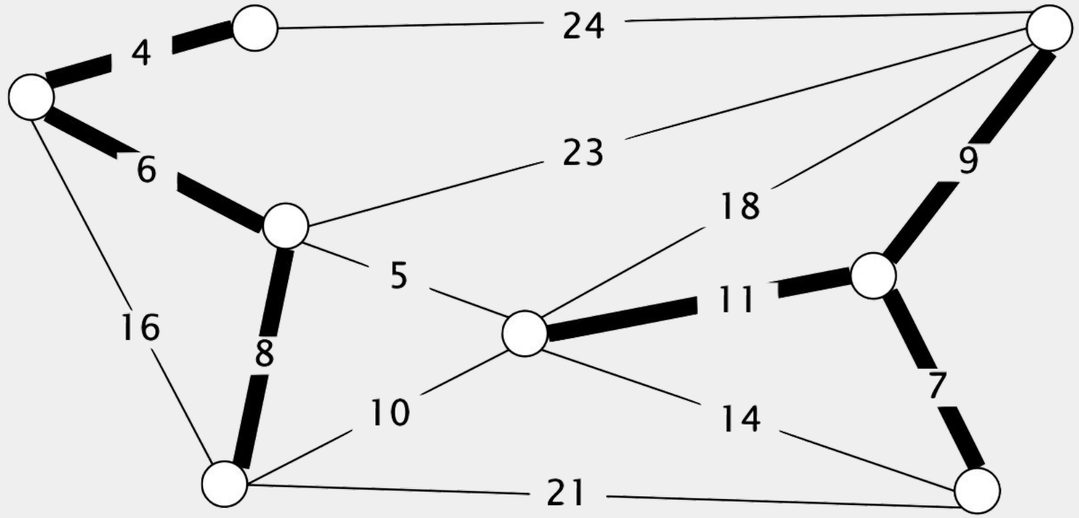
Figura 1 - Grafo não direcionado, ponderado, conexo e acíclico: O grafo G serve como base para o problema da Árvore Geradora Mínima (MST). O objetivo é encontrar um subgrafo gerador T que seja uma árvore e que inclua todos os vértices, minimizando o peso total. Fonte: SEDGEWICK, 2017.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

1. Árvore Geradora Mínima - MST

1.1 Introdução



Grafo G não conectado

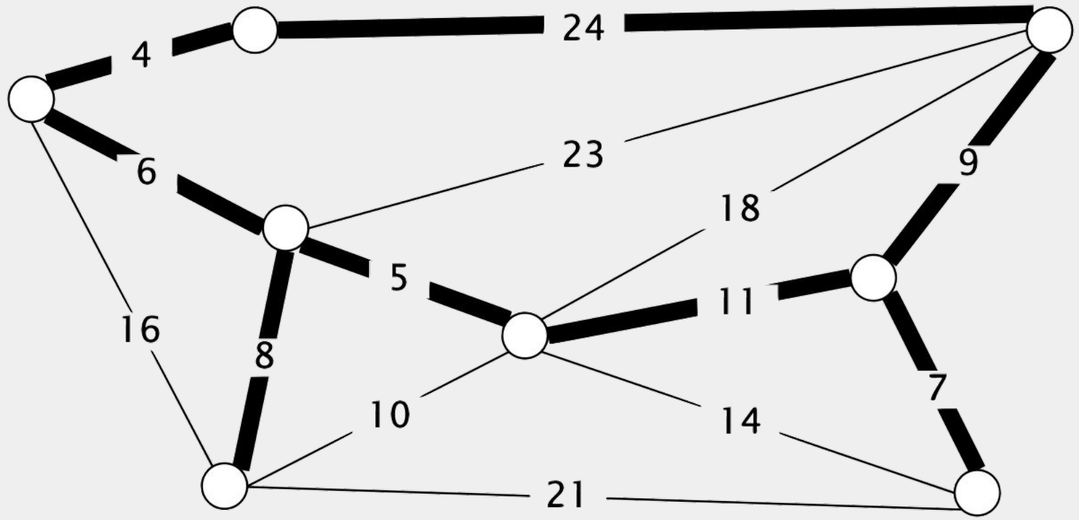
Figura 2 - Subgrafo não conectado, ponderado e acíclico:Em uma MST, o subgrafo resultante deve ser acíclico, conectado e incluir exatamente V−1 arestas com o menor custo total possível. Portanto, o grafo apresentado não representa uma MST. Fonte: SEDGEWICK, 2017.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

1. Árvore Geradora Mínima - MST

1.1 Introdução



Grafo G não é acíclico

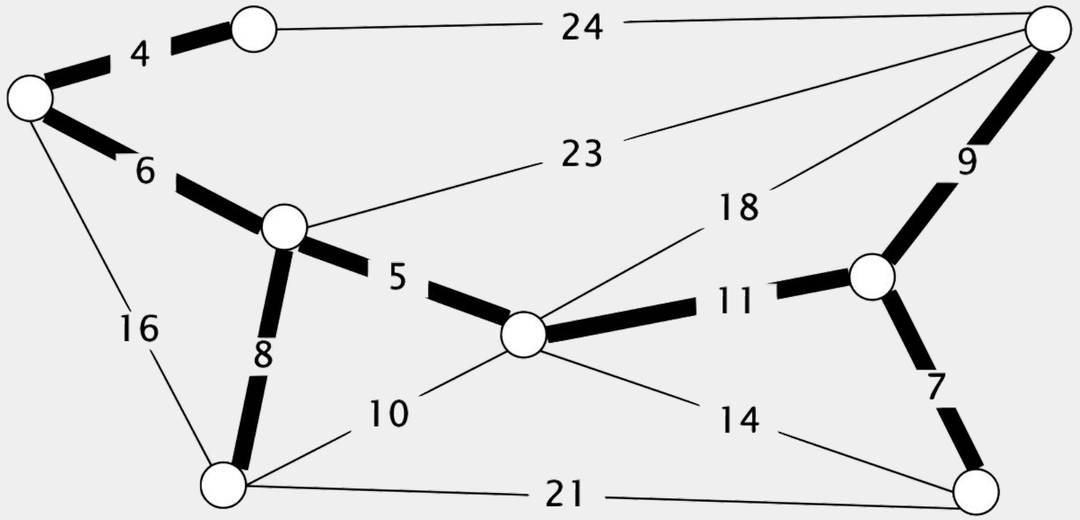
Figura 3 - Subgrafo conectado, ponderado e não acíclico:Embora conecte todos os vértices, este subgrafo não representa uma MST, pois contém ciclos e mais do que V−1 arestas, o que viola as propriedades fundamentais de uma árvore geradora Fonte: SEDGEWICK, 2017.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

1. Árvore Geradora Mínima - MST

1.1 Introdução



Árvore Geradora Mínima (MST) T: custo = 50 = 4 + 6 + 8 + 5 + 11 + 9 + 7

Figura 4 - Subgrafo não direcionado, ponderado, conexo e acíclico: O subgrafo representado pelas arestas espessas correspondem às de menor custo entre os pares de vértices, e conectam todos os vértices sem formar ciclos, com um custo total mínimo de 50. Fonte: SEDGEWICK, 2017.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

1. Árvore Geradora Mínima – MST

1.2 Aplicações

MST é um problema fundamental com diversas aplicações.

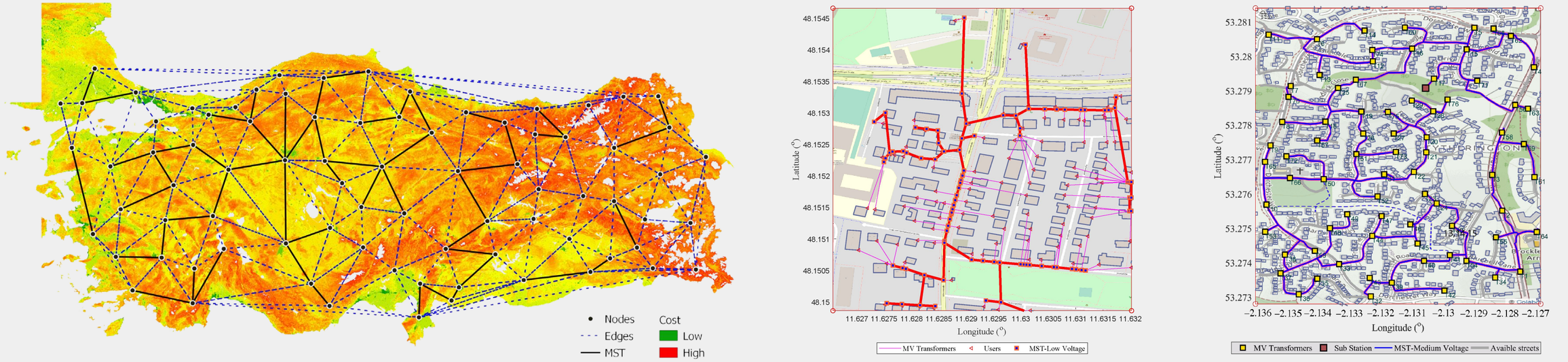
- ***Dithering**: distribuição eficiente de pontos em imagens (Dither Me This).*
- ***Verificação facial** em tempo real: extração de características.*
- ***Detecção de redes viárias**: extração em imagens de satélite e aéreas.*
- ***Spanning Tree Protocol (STP)**: evita ciclos em redes locais (LANs), desativando temporariamente conexões redundantes e mantendo a conectividade total.*
- ***Aproximação em NP-difíceis**: algoritmos relacionados a TSP e árvore de Steiner.*
- ***Projeto de redes**: determinam a forma mais econômica de conectar diferentes pontos em uma rede de comunicação, elétrica, hidráulica, computacional ou viária.*

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

1. Árvore Geradora Mínima

1.2 Aplicações



(a) **(b)** **(c)**

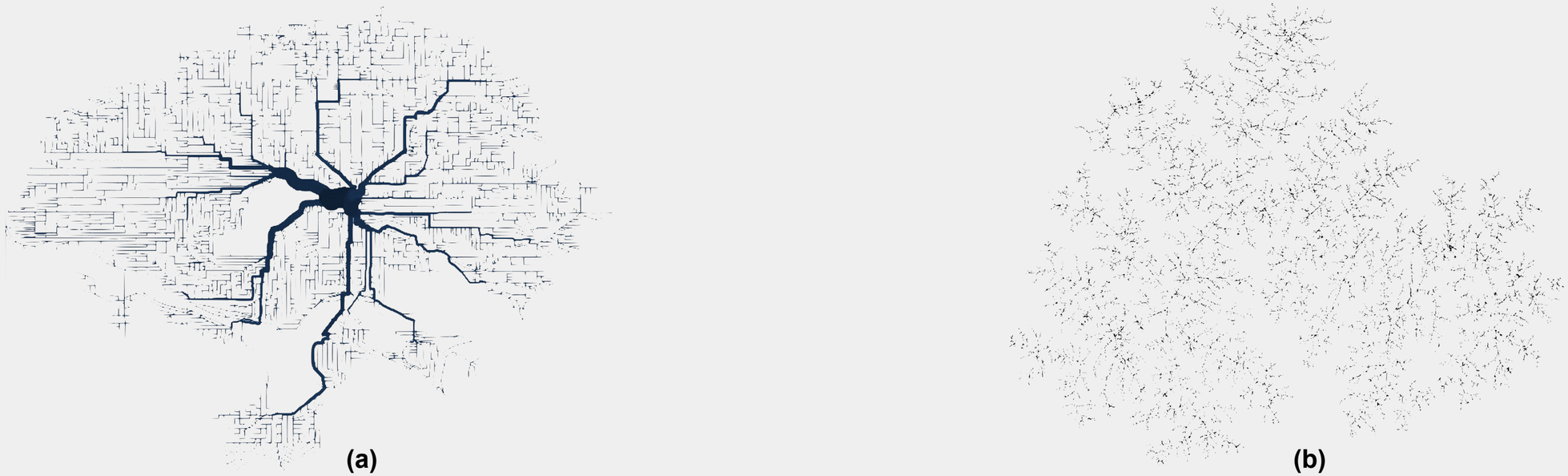
Figura 5 - Exemplos de aplicação de Árvore Geradora Mínima (MST) em problemas espaciais. (a) MST para minimização do custo de conexão entre cidades considerando relevo, declividade, cobertura do solo e restrições geográficas[1]. (b) MST para otimização da conexão entre transformadores em uma rede elétrica urbana na Alemanha. (c) MST para otimização da conexão entre transformadores em uma rede elétrica urbana no Reino Unido[2].

Notas:

1. ÇALIŞKAN, M.; ANBAROĞLU, B. Geo-MST: A geographical minimum spanning tree plugin for QGIS. SoftwareX, v. 12, 100553, 2020. DOI: 10.1016/j.softx.2020.100553.
2. PAVON, W.; TORRES, M.; INGA, E. Integrating Minimum Spanning Tree and MILP in Urban Planning: A Novel Algorithmic Perspective. Buildings, v. 14, 213, 2024. DOI: 10.3390/buildings14010213.

1. Árvore Geradora Mínima - MST

1.2 Aplicações



(a) (b)

Figura 5 - Árvores Geradoras Mínima (MST) de (a) rotas de bicicleta no norte de Seattle e de (b) um grafo aleatório. Em (a), as linhas representam a rede otimizada que conecta todos os pontos da malha cicloviária com o menor custo total e sem ciclos, destacando os principais corredores de deslocamento. (b) MST de um grafo aleatório que forma padrões ramificados semelhantes aos observados em sistemas naturais. Fonte: SEDGEWICK, 2017.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

1. Árvore Geradora Mínima – MST

1.3 Estrategia gulosa

Premissas:

- *Os pesos das arestas são distintos.*
- *O grafo é conexo.*

Consequência: *A MST existe e é única.*

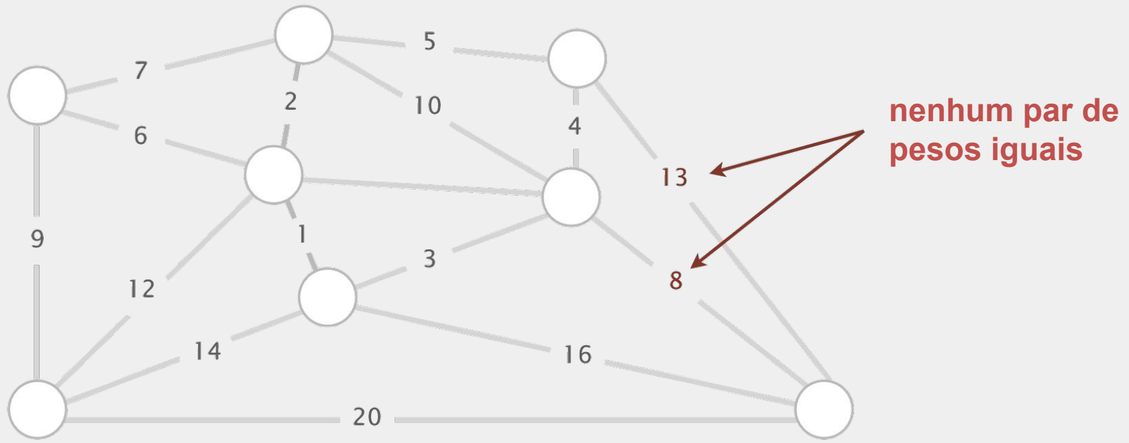


Figura 7 – Grafo conexo com pesos distintos nas arestas. Conforme as premissas simplificadas, a Árvore Geradora Mínima (MST) é garantida e únic.
Fonte: SEDGEWICK, 2017.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

1. Árvore Geradora Mínima – MST

1.3 Estrategia gulosa

Def.: *Corte num grafo é uma partição de vértices em dois subconjuntos disjuntos não vazios.*

Def.: *Uma aresta de corte é uma aresta que conecta um vértice de um conjunto a um vértice do outro.*

Propriedade: *Dado qualquer corte em um grafo conexo com pesos distintos nas arestas, a aresta de corte de menor peso pertence à árvore geradora mínima (MST).*

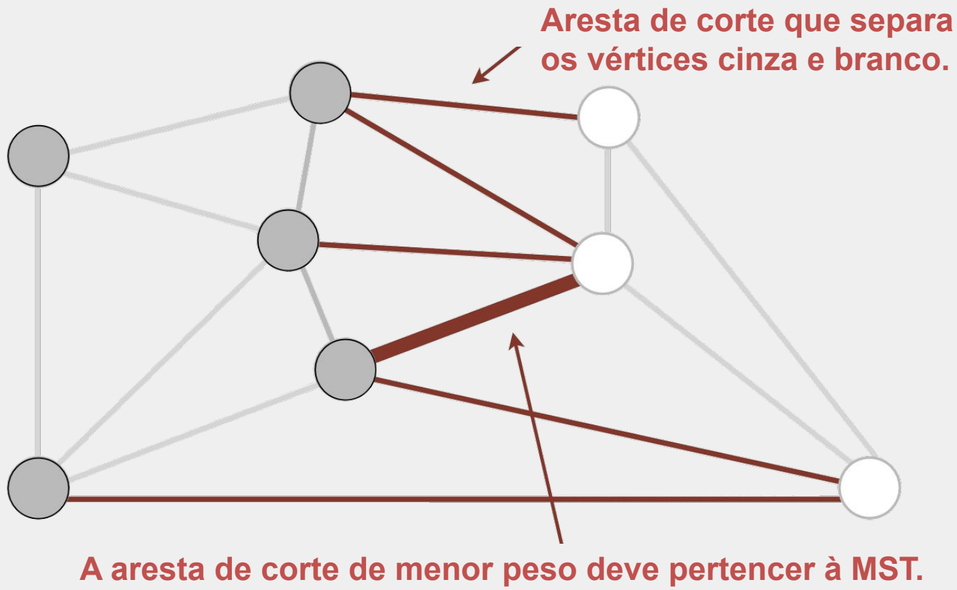


Figura 8 – Propriedade do corte em grafos. Os vértices estão particionados em dois subconjuntos: cinza e branco. As arestas vermelhas representam as arestas de corte, e a aresta mais espessa é aresta de corte com o menor peso, que, segundo a propriedade, deve pertencer à MST. Fonte: SEDGEWICK, 2017.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

1. Árvore Geradora Mínima – MST

1.3 Estrategia gulosa

Prova (por contradição): *Suponha que a aresta de corte de menor peso e não pertence à MST.*

- *Adicionar a aresta e à MST cria um ciclo.*
- *Alguma outra aresta f desse ciclo também deve ser uma aresta de corte.*
- *Remover f e adicionar e e ainda resulta em uma árvore geradora.*
- *Como o peso de e é menor que o de f, a nova árvore tem peso total menor.*

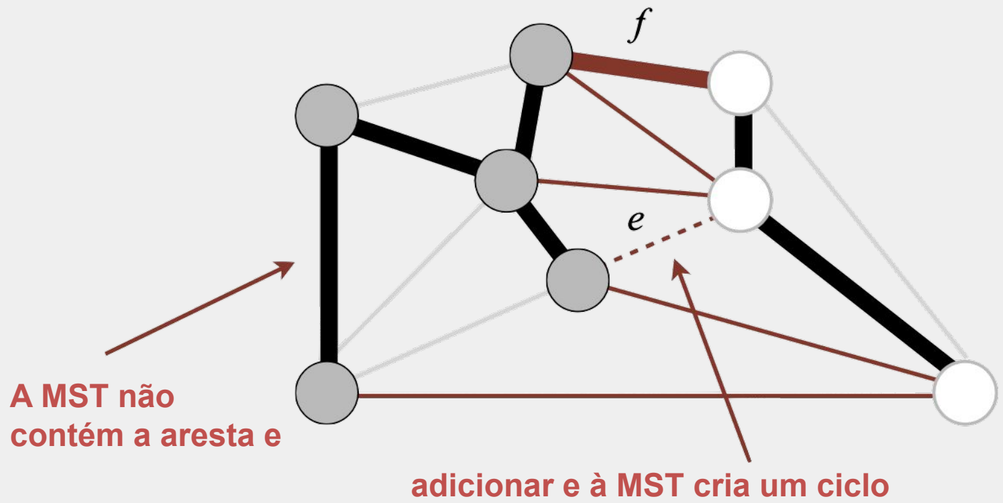


Figura 9 – Prova da propriedade do corte em MSTs. A adição da aresta e, que não pertence à MST, gera um ciclo no grafo. Pela propriedade do corte, a troca de f por e resultaria em uma árvore geradora de peso menor, contradizendo a premissa. Fonte: SEDGEWICK, 2017.

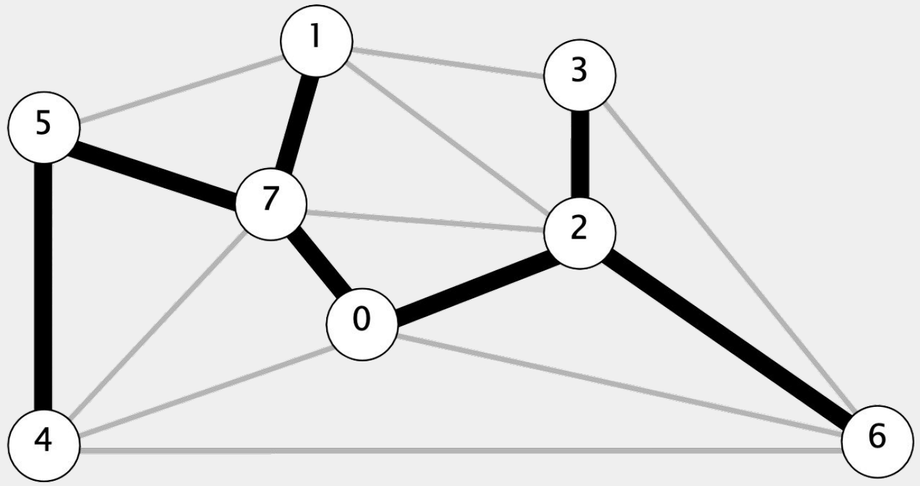
Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

1. Árvore Geradora Mínima – MST

1.4 Algoritmo guloso

Passo 1: *Comece com todas as arestas coloridas de cinza.*
Passo 2: *Encontre um corte sem arestas pretas; pinte de preto a aresta de corte de menor peso.*
Passo 3: *Repita até que V−1 arestas estejam coloridas de preto.*



MST 0-2 5-7 6-2 0-7 2-3 1-7 4-5

Figura 10 – Construção da MST. As arestas espessas forma a MST conforme os passos do algoritmo guloso descrito. Fonte: SEDGEWICK, 2017.

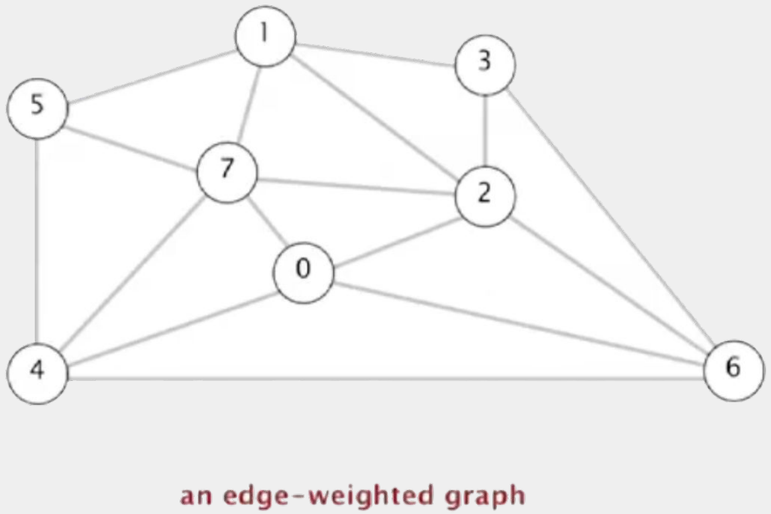
Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

Universidade de Fortaleza

1. Árvore Geradora Mínima – MST

1.4 Algoritmo guloso



0-7	0.16	3-6	0.52
2-3	0.17	6-0	0.58
1-7	0.19	6-4	0.93
0-2	0.26		
5-7	0.28		
1-3	0.29		
1-5	0.32		
2-7	0.34		
4-5	0.35		
1-2	0.36		
4-7	0.37		
0-4	0.38		
6-2	0.40		

Figura 11 – Grafo ponderado utilizado para ilustrar o algoritmo guloso de construção da Árvore Geradora Mínima (MST). Todas as arestas começam cinza; em cada passo, seleciona-se um corte sem arestas pretas cruzando e pinta-se de preto a aresta de menor peso que cruza esse corte. O processo se repete até que (V - 1) arestas estejam pretas. Fonte: SEDGEWICK, 2017.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

Universidade de Fortaleza

1. Árvore Geradora Mínima - MST

1.4 Algoritmo guloso

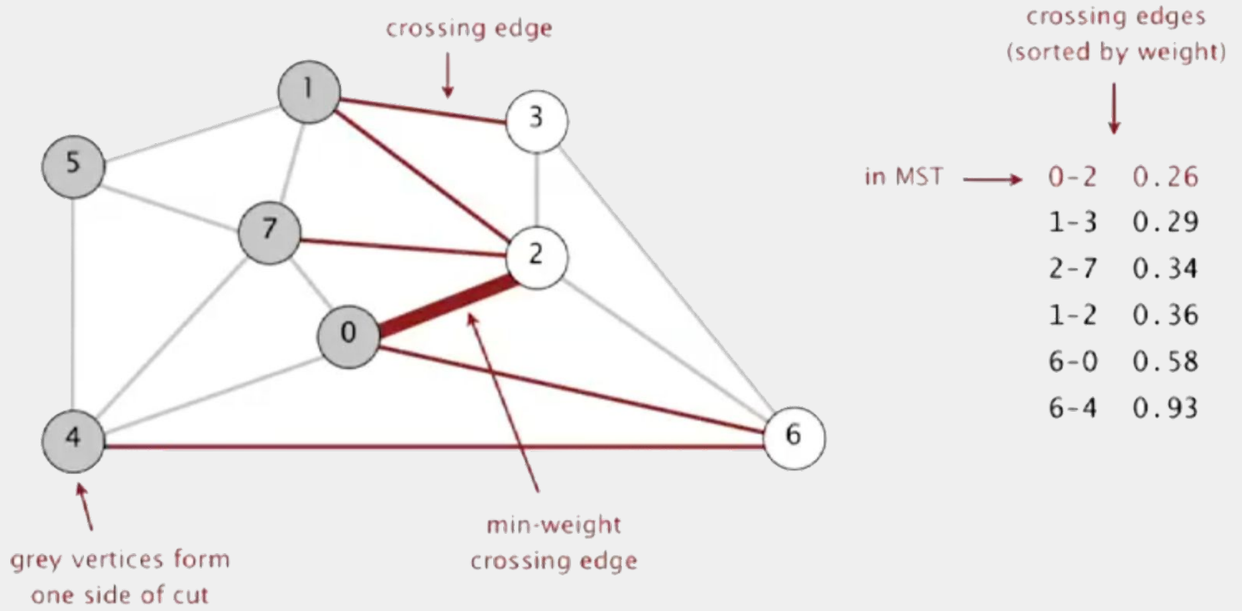


Figura 12 – Primeira iteração do algoritmo guloso para construção da Árvore Geradora Mínima (MST). Os vértices cinza formam um lado do corte ({0, 1, 4, 5, 7}) e os vértices brancos o outro ({2, 3, 6}). As arestas de corte identificadas são: 0–2 (0.26), 1–3 (0.29), 2–7 (0.34), 1–2 (0.36), 6–0 (0.58) e 6–4 (0.93). A aresta de menor peso entre elas, 0–2, é selecionada e incorporada à MST. Fonte: SEDGEWICK, 2017.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

1. Árvore Geradora Mínima – MST

1.4 Algoritmo guloso

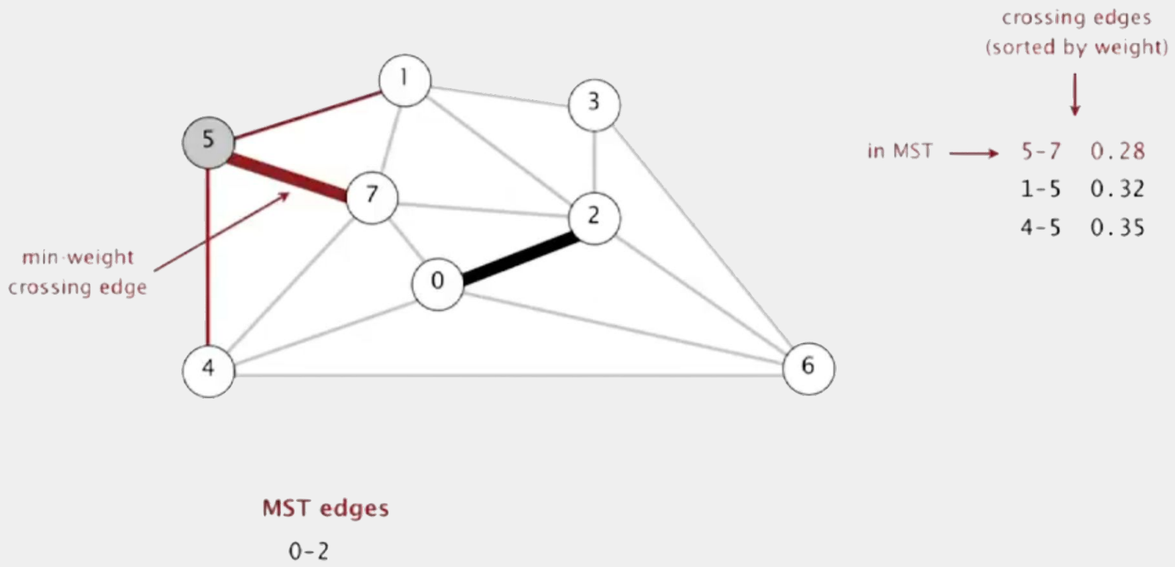


Figura 13 – Segunda iteração do algoritmo guloso para construção da Árvore Geradora Mínima (MST). Os vértices cinza formam o lado esquerdo do corte ({5}) e os vértices brancos o lado direito ({0, 1, 2, 3, 4, 6, 7}). As arestas de corte são: 5–7 (0.28), 1–5 (0.32), 4–5 (0.35). A aresta de menor peso, 5–7, é selecionada e incorporada à MST. Fonte: SEDGEWICK, 2017.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

1. Árvore Geradora Mínima - MST

1.4 Algoritmo guloso

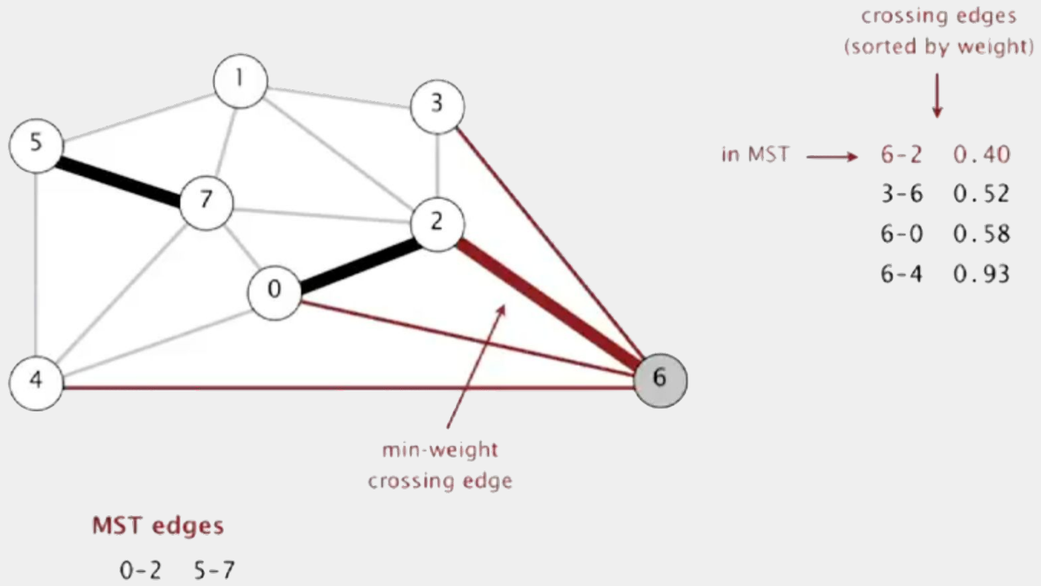


Figura 14 – Terceira iteração do algoritmo guloso para construção da Árvore Geradora Mínima (MST). O vértice cinza {6} forma um lado do corte e os demais vértices brancos o outro lado. As arestas de corte são: 6–2 (0.40), 3–6 (0.52), 6–0 (0.58), 6–4 (0.93). A aresta de menor peso, 6–2, é selecionada e incorporada à MST. Fonte: SEDGEWICK, 2017.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

1. Árvore Geradora Mínima - MST

1.4 Algoritmo guloso

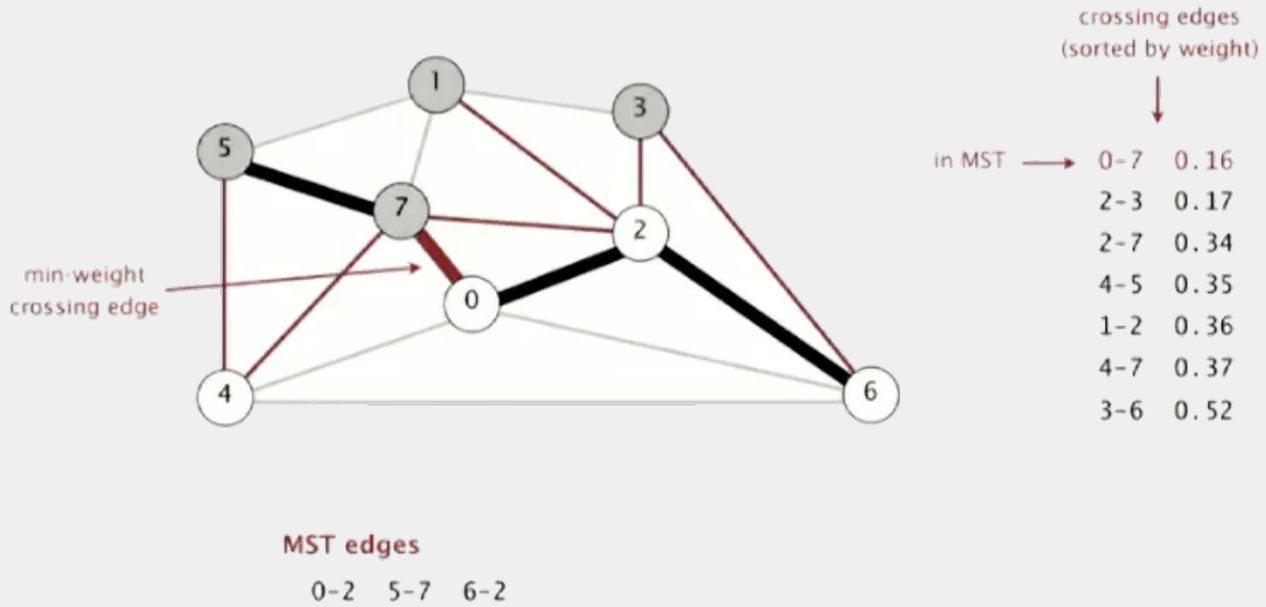


Figura 15 – Quarta iteração do algoritmo guloso para construção da Árvore Geradora Mínima (MST). Os vértices cinza formam o lado esquerdo do corte ({0, 2, 5, 6, 7}) e os vértices brancos o lado direito ({1, 3, 4}). As arestas de corte são: 0–7 (0.16), 2–3 (0.17), 2–7 (0.34), 4–5 (0.35), 1–2 (0.36), 4–7 (0.37), 3–6 (0.52). A aresta de menor peso, 0–7, é selecionada e incorporada à MST. Fonte: SEDGEWICK, 2017.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

1. Árvore Geradora Mínima - MST

1.4 Algoritmo guloso

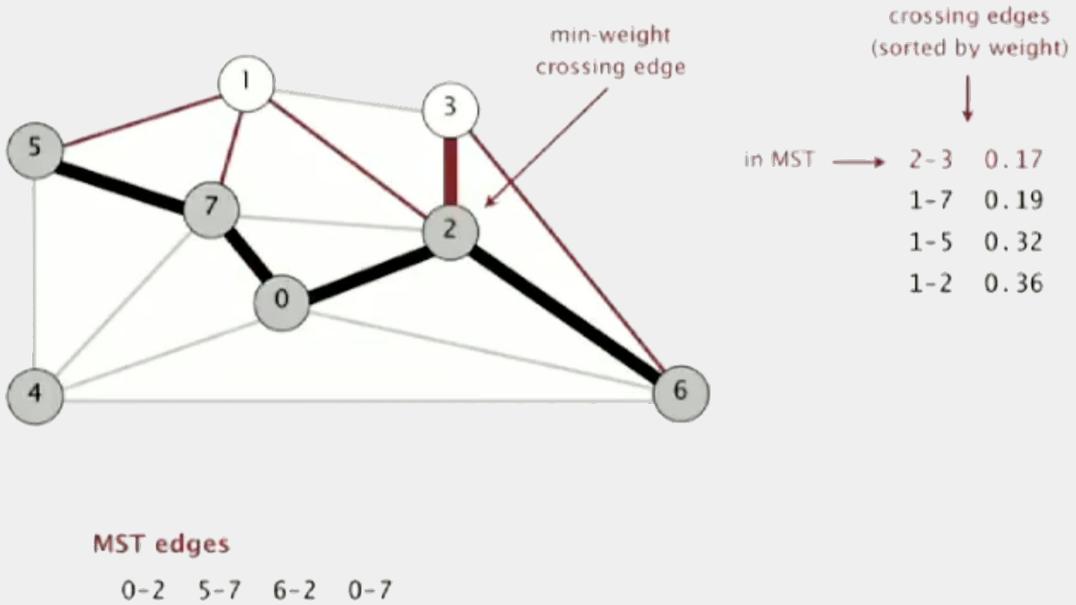


Figura 16 – Quinta iteração do algoritmo guloso para construção da Árvore Geradora Mínima (MST). Os vértices cinza formam o lado esquerdo do corte ({0, 2, 4, 5, 6, 7}) e os vértices brancos o lado direito ({1, 3}). As arestas de corte são: 2–3 (0.17), 1–7 (0.19), 1–5 (0.32), 1–2 (0.36). A aresta de menor peso, 2–3 (0.17), é incluída na MST. Fonte: SEDGEWICK, 2017.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

1. Árvore Geradora Mínima - MST

1.4 Algoritmo guloso

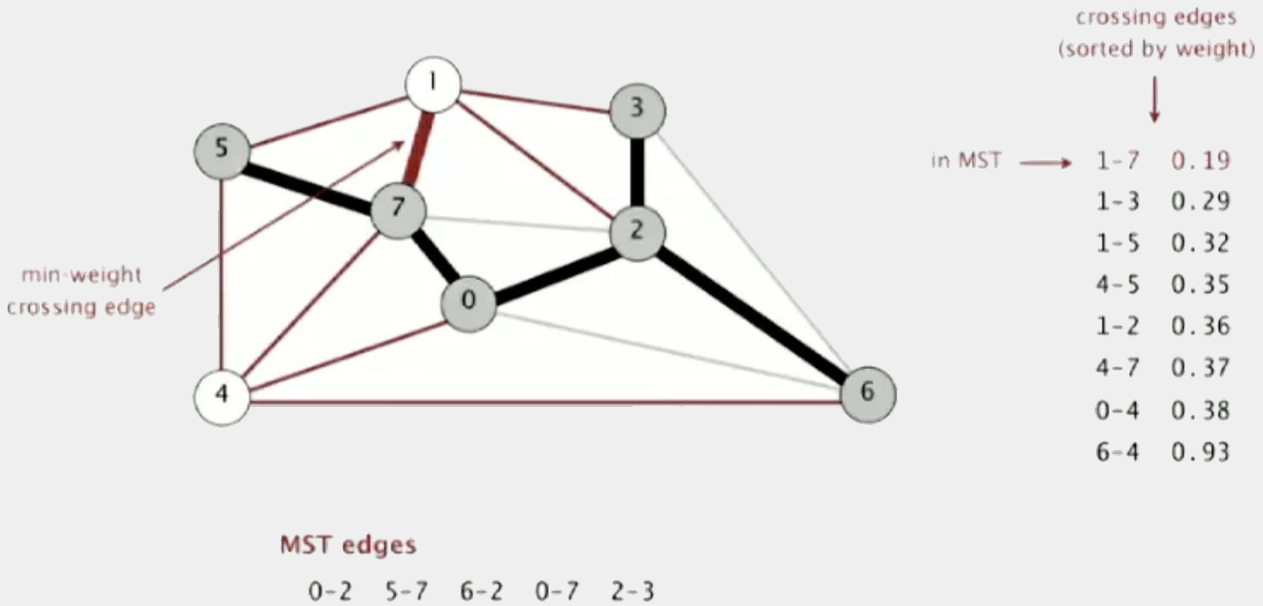


Figura 17 – Sexta iteração do algoritmo guloso para construção da Árvore Geradora Mínima (MST). Os vértices cinza formam o lado esquerdo do corte ({0, 2, 3, 5, 6, 7}) e o vértice branco o lado direito ({1,4}). As arestas de corte são: 1–7 (0.19), 1–3 (0.29), 1–5 (0.32), 1–2 (0.36), 4-7 (0.37). A aresta de menor peso, 1–7 (0.19), é incluída na MST. Fonte: SEDGEWICK, 2017.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

1. Árvore Geradora Mínima - MST

1.4 Algoritmo guloso

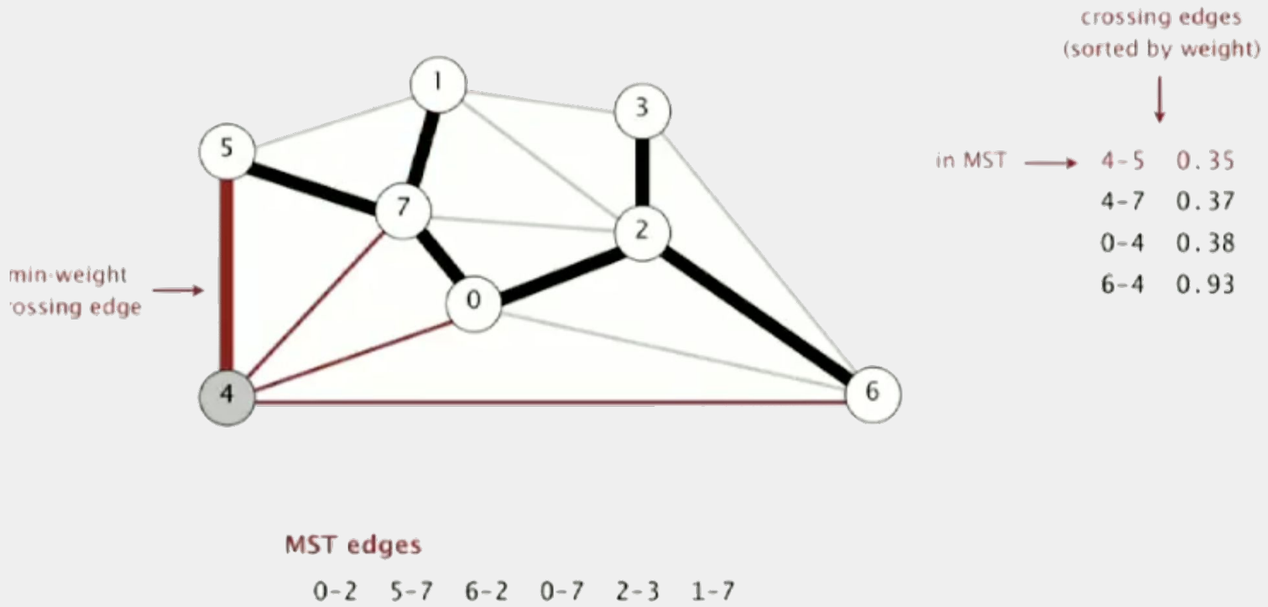


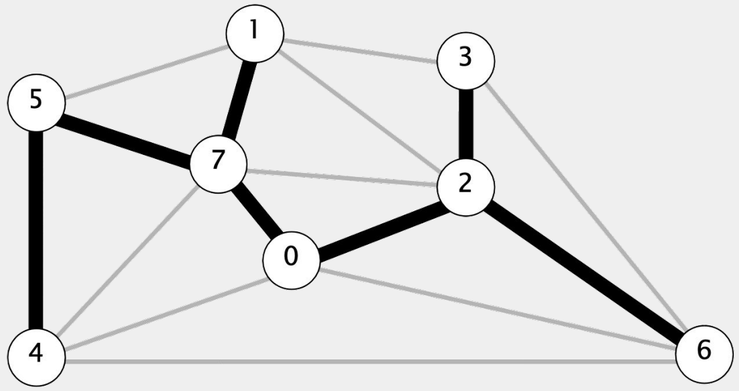
Figura 18 – Sétima iteração do algoritmo guloso para construção da Árvore Geradora Mínima (MST). O vértice cinza forma o lado esquerdo do corte ({4}) e os vértices brancos o lado direito ({0, 1, 2, 3, 5, 6, 7}). As arestas de corte são: 4–5 (0.35), 4–7 (0.37), 0–4 (0.38), 6–4 (0.93). A aresta de menor peso, 4–5 (0.35), é incluída na MST. Fonte: SEDGEWICK, 2017.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

1. Árvore Geradora Mínima - MST

1.4 Algoritmo guloso



MST edges

0-2 5-7 6-2 0-7 2-3 1-7 4-5

Figura 19 – Árvore Geradora Mínima (MST) finalizada após a aplicação do algoritmo guloso. As arestas selecionadas: 0–2 (0.26), 5–7 (0.28), 6–2 (0.40), 0–7 (0.16), 2–3 (0.17), 1–7 (0.19) e 4–5 (0.35), totalizando 7 arestas conectando todos os 8 vértices. Limitações: (i) requer que todos os pesos das arestas sejam distintos; (ii) não considera múltiplas soluções; e (iii) é sensível à escolha da estrutura de dados para os cortes e as arestas. Fonte: SEDGEWICK, 2017.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

1. Árvore Geradora Mínima – MST

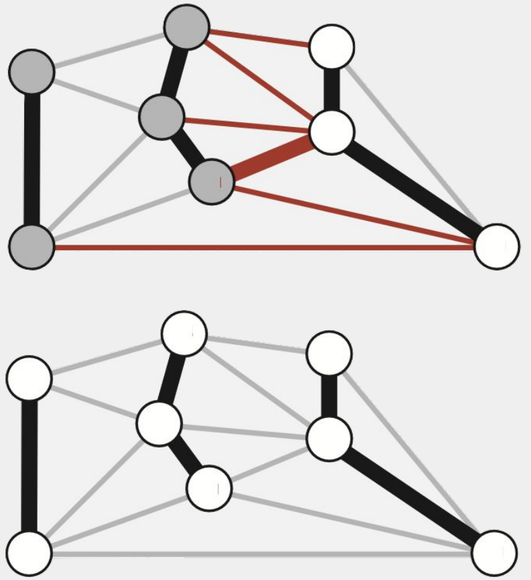
1.5 Prova de Corretude

Proposição: *O algoritmo guloso computa a MST.*

Prova:

(1) *Qualquer aresta preta está na MST (princípio do corte adiciona arestas de corte com menor peso).*

(2) *Menos de V − 1 arestas pretas ⇒ existe um corte sem arestas de corte pretas (considere o corte cujos vértices formam um componente conexo).*



(a) existe um corte sem arestas de corte pretas

(b) Menos de V − 1 arestas pretas

Figura 20 – Prova de corretude. (a) existe um corte sem arestas pretas cruzando; (b) há menos de V − 1 arestas pretas, logo o algoritmo ainda não terminou. Fonte: SEDGEWICK, 2017.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

Universidade de Fortaleza

1. Árvore Geradora Mínima - MST

1.7 Implementações Eficientes

Proposição. *O algoritmo guloso calcula a Árvore Geradora Mínima (MST).*

Estratégia*: Como escolher o corte? Como encontrar a aresta de menor peso?*

(1) *Algoritmo de Kruskal: O(E log V) usando Union-Find;*
(2) *Algoritmo de Prim: O(E log V) usando Heap Binária;*
(3) *Algoritmo de Borůvka: O(E log V).*



(a) (b)

Figura 21 – Referências de pesquisa. (a) VisuAlgo ilustra o funcionamento da Union Find (DSU), além do canal Willian Fiset com introdução DSU, operações, compressão de caminho e implementação (b) Departamento de Matemática. Escola de Computação, Informação e Tecnologia da TUM. Universidade Técnica de Munique

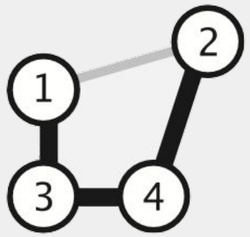
Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

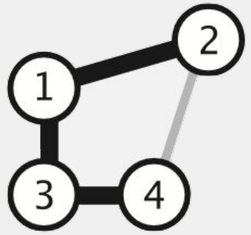
1. Árvore Geradora Mínima – MST

1.7 Removendo as premissas limitantes

Ṗ. *E se os pesos das arestas não forem todos distintos?*
R. *A MST pode não ser única quando há pesos iguais. O algoritmo guloso para MST ainda é correto mesmo com pesos iguais!*




```
1 2  1.00
1 3  0.50
2 4  1.00
3 4  0.50
```




```
1 2  1.00
1 3  0.50
2 4  1.00
3 4  0.50
```

Figura 22 – Exemplo de grafo com pesos não distintos. Existem duas Árvores Geradoras Mínimas (MSTs) distintas com o mesmo custo total (2.00). O algoritmo guloso pode produzir qualquer uma delas dependendo da ordem de escolha das arestas com pesos iguais. Fonte: SEDGEWICK, 2017.

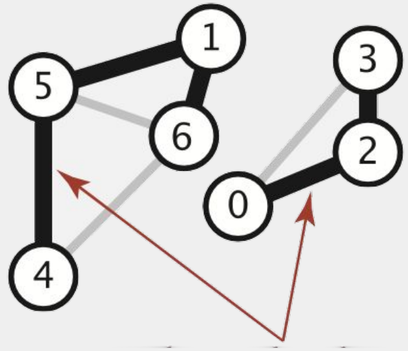
Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

1. Árvore Geradora Mínima – MST

1.7 Removendo as premissas limitantes

P. *E se o grafo não for conexo?*
R. *Devemos calcular uma floresta geradora mínima, ou seja, a MST de cada componente conexo.*




```
4 5  0.61
4 6  0.62
5 6  0.88
1 5  0.11
2 3  0.35
0 3  0.6
1 6  0.10
0 2  0.22
```

calcula independentemente
as MSTs das componentes

Figura 23 – O grafo possui duas componentes desconexas. O algoritmo guloso constrói uma floresta geradora mínima (minimum spanning forest), aplicando a estratégia de menor peso em cada componente separadamente. Arestas selecionadas: {1–6 (0.10), 1–5 (0.11), 4–5 (0.61)} e {0–2 (0.22), 2–3 (0.35)}. Fonte: SEDGEWICK, 2017.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

Universidade de Fortaleza

1. Árvore Geradora Mínima – MST

1.8 API de arestas ponderadas

public class Edge implements Comparable<Edge>	
Edge(int v, int w, double weight)	*create a weighted edge v-w*
int either()	*either endpoint*
int other(int v)	*the endpoint that's not v*
int compareTo(Edge that)	*compare this edge to that edge*
double weight()	*the weight*
String toString()	*string representation*

A classe **Edge** modela uma aresta entre dois vértices v e w, com peso real associado (`weight`).

O método **`either()`** retorna um dos extremos da aresta, e **`other(v)`** retorna o vértice oposto ao fornecido.

O método **`compareTo(Edge that)`** permite a comparação entre arestas com base em seus pesos, essencial para ordenação em algoritmos como Kruskal.

Os métodos **`weight()`** e **`toString()`** retornam, respectivamente, o valor do peso e a representação textual da aresta.

Expressão idiomática para processar uma aresta **e: int v = e.either(), w = e.other(v).** Escolha um dos extremos da aresta com either(), e o outro vértice com other(v). Essa construção é usada para percorrer os dois extremos de uma aresta sem se preocupar com a ordem.

Figura 24 – API da classe Edge para grafos ponderados não direcionados.. Define operações básicas para criação, acesso e comparação de arestas com peso, fundamentais na construção de árvores geradoras mínimas.Fonte: SEDGEWICK, 2017.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

Universidade de Fortaleza

1. Árvore Geradora Mínima – MST

1.8 Implementação Java da classe Edge

```
public class Edge implements Comparable<Edge>
{
   private final int v, w;
   private final double weight;

   public Edge(int v, int w, double weight)
   {
      this.v = v;
      this.w = w;
      this.weight = weight;
   }

   public int either()
   {  return v;  }
```

Define a classe **Edge**.A interface **Comparable<Edge>** indica que objetos dessa classe podem ser comparados entre si (ordenar arestas pelo peso).

v e **w**: vértices extremos da aresta. **weight**: peso da aresta. Todos são **final**, indicando que não podem ser modificados após inicialização.

Construtor. Inicializa a aresta com dois vértices (**v** e **w**) e um peso **weight**.

Método **either**(). Retorna um dos extremos da aresta, aqui arbitrariamente o v. Útil para começar a explorar a aresta a partir de um vértice.

Figura 25 – Implementação Java da classe `Edge` para arestas ponderadas. Define uma aresta com dois extremos e um peso associado, incluindo métodos para acessar os vértices e comparar arestas com base no peso. Fonte: SEDGEWICK, 2017.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

Universidade de Fortaleza

1. Árvore Geradora Mínima – MST

1.8 Implementação Java da classe Edge

```
   public int other(int vertex)
   {
      if (vertex == v) return w;
      else return v;
   }

   public int compareTo(Edge that)
   {
      if      (this.weight < that.weight) return -1;
      else if (this.weight > that.weight) return +1;
      else                                return  0;
   }
}
```

Método **other(int vertex)**. Dado um vértice **vertex** da aresta, retorna o outro extremo.Se **vertex** for v, retorna **w**; caso contrário, retorna **v**. Garante navegação em grafos a partir de um vértice conhecido.

Método **compareTo**(Edge that). Compara duas arestas com base no peso. Retorna -1 se esta aresta possuir um menor peso que a outra, +1 se for maior, 0 se tiverem o mesmo peso. Essencial para ordenação e uso em estruturas como **PriorityQueue**.

Figura 26 – Implementação Java da classe `Edge` para arestas ponderadas. Define uma aresta com dois extremos e um peso associado, incluindo métodos para acessar os vértices e comparar arestas com base no peso. Fonte: SEDGEWICK, 2017.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

Universidade de Fortaleza

1. Árvore Geradora Mínima – MST

1.9 API de grafos ponderados por arestas

public class EdgeWeightedGraph	
EdgeWeightedGraph(int V)	*create an empty graph with V vertices*
EdgeWeightedGraph(In in)	*create a graph from input stream*
void addEdge(Edge e)	*add weighted edge e to this graph*
Iterable<Edge> adj(int v)	*edges incident to v*
Iterable<Edge> edges()	*all edges in this graph*
int V()	*number of vertices*
int E()	*number of edges*
String toString()	*string representation*

Define a classe **EdgeWeightedGraph**, usada para representar um grafo com arestas ponderadas.

Construtor que cria um grafo vazio com V vértices e nenhuma aresta.

Construtor que cria o grafo a partir de uma entrada de dados.

Adiciona uma aresta ponderada e ao grafo. Essa aresta conecta dois vértices e possui peso associado.

Retorna uma coleção iterável de arestas incidentes ao vértice v (ou seja, todas as arestas conectadas a v).

Retorna um iterável com todas as arestas do grafo (não apenas as de um vértice específico).

Retorna o número de vértices, de arestas e uma representação textual do grafo, geralmente listando suas arestas adjacentes.

Convenções. Permite loops e arestas paralelas.

Figura 27 – Interface da classe `EdgeWeightedGraph`. Define a API para representar grafos ponderados por arestas. Inclui métodos para construir o grafo, adicionar arestas, recuperar arestas adjacentes a um vértice e acessar propriedades globais como número de vértices e arestas. Fonte: SEDGEWICK, 2017.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

1. Árvore Geradora Mínima – MST

1.10 Grafo ponderado: Representação por listas de adjacência

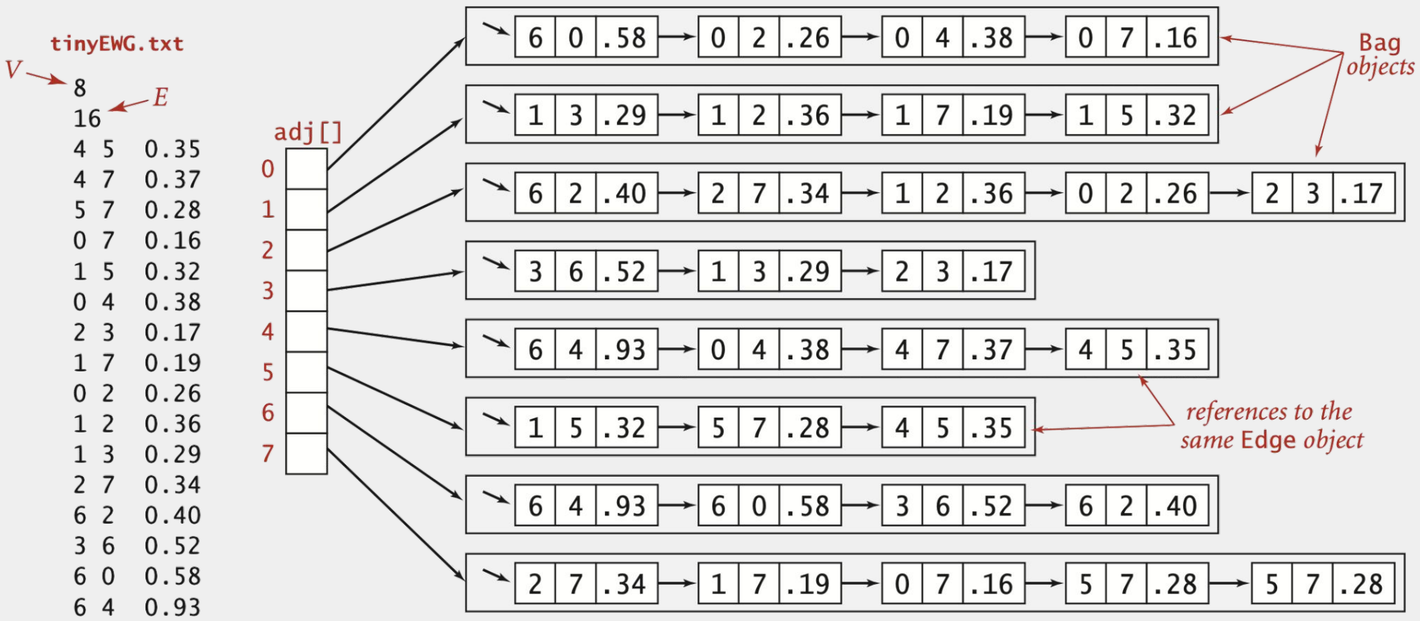


Figura 28 – Representação de um grafo ponderado com 8 vértices e 16 arestas usando listas de adjacência. Cada posição do vetor adj[] armazena uma lista (Bag) com as arestas incidentes a um vértice. Fonte: SEDGEWICK, 2017

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

Universidade de Fortaleza

1. Árvore Geradora Mínima – MST

1.11 Grafo ponderado: implementação com listas de adjacência

```
public class EdgeWeightedGraph
{
   private final int V;
   private final Bag<Edge>[] adj;

   public EdgeWeightedGraph(int V)
   {
      this.V = V;
      adj = (Bag<Edge>[]) new Bag[V];
      for (int v = 0; v < V; v++)
         adj[v] = new Bag<Edge>();
   }
```

Campo privado V, que armazena o número de vértices do graf

Vetor **adj[]** de bolsas (**Bag<Edge>**), onde cada índice representa um vértice, e a bolsa armazena as arestas incidentes a esse vértice.

Construtor que recebe o número de vértices do grafo.

Atribui o número de vértices à variável de instância.

Cria um vetor de bolsas, com uma bolsa para cada vértice

Inicializa cada posição do vetor com uma nova instância de **Bag<Edge>**

Figura 29 – Estrutura de um grafo ponderado representado por listas de adjacência. Cada vértice possui uma lista que armazena as arestas conectadas a ele. Como o grafo é não direcionado, cada aresta é armazenada em ambos os vértices que ela conecta. Fonte: SEDGEWICK, 2017.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

Universidade de Fortaleza

1. Árvore Geradora Mínima – MST

1.11 Grafo ponderado: implementação com listas de adjacência

```
   public void addEdge(Edge e)
   {
      int v = e.either(), w = e.other(v);
      adj[v].add(e);
      adj[w].add(e);
   }

   public Iterable<Edge> adj(int v)
   {  return adj[v];  }
}
```

Método público que adiciona uma aresta ponderada e ao grafo.
Obtém os dois vértices da aresta e: um usando either() e o outro usando other(v).
Adiciona a aresta às bolsas dos dois vértices, pois é um grafo não direcionado: ambos os vértices devem referenciar essa aresta.

Método que retorna todas as arestas incidentes ao vértice v.
Retorna a bolsa de arestas que estão conectadas ao vértice v.

Figura 30 – Estrutura de um grafo ponderado representado por listas de adjacência. Cada vértice possui uma lista que armazena as arestas conectadas a ele. Como o grafo é não direcionado, cada aresta é armazenada em ambos os vértices que ela conecta. Fonte: SEDGEWICK, 2017.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

Universidade de Fortaleza

1. Árvore Geradora Mínima – MST

1.12 API da Árvore Geradora Mínima: Como representar a MST?

public class MST	
MST(EdgeWeightedGraph G)	*constructor*
Iterable<Edge> edges()	*edges in MST*
double weight()	*weight of MST*

Define uma classe chamada **MST** que representará a árvore geradora mínima de um grafo ponderado.

Construtor da classe. Recebe um grafo ponderado G como parâmetro e constrói sua árvore geradora mínima. Internamente, aplicará um algoritmo de MST (como Kruskal ou Prim).

Método que retorna todas as arestas que compõem a árvore geradora mínima. Essas arestas pertencem a um subconjunto do grafo original.

Método que retorna o peso total da árvore geradora mínima, ou seja, a soma dos pesos das arestas retornadas por edges().

Figura 31 – API da classe MST. Estrutura para representar a Árvore Geradora Mínima (MST) de um grafo ponderado. Fonte: SEDGEWICK, 2017.

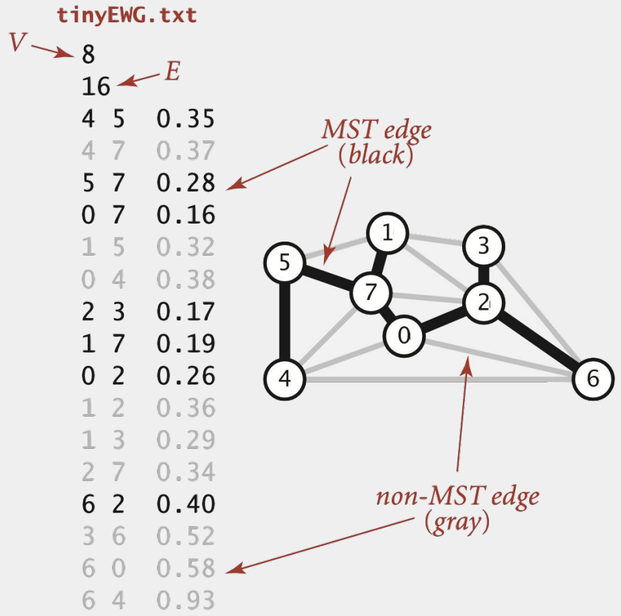
Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

Universidade de Fortaleza

1. Árvore Geradora Mínima – MST

1.12 API da Árvore Geradora Mínima: Como representar a MST?




```
public static void main(String[] args)
{
   In in = new In(args[0]);
   EdgeWeightedGraph G = new EdgeWeightedGraph(in);
   MST mst = new MST(G);
   for (Edge e : mst.edges())
      StdOut.println(e);
   StdOut.printf("%.2f\n", mst.weight());
}
```

```
% java MST tinyEWG.txt
0-7 0.16
1-7 0.19
0-2 0.26
2-3 0.17
5-7 0.28
4-5 0.35
6-2 0.40
1.81
```

Figura 32 – Exemplo de uso da API para construir e exibir uma MST a partir de um grafo ponderado lido de um arquivo A visualização destaca as arestas da MST (em preto) e as demais (em cinza). A execução do programa imprime as arestas da MST e seu peso total. Fonte: SEDGEWICK, 2017

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

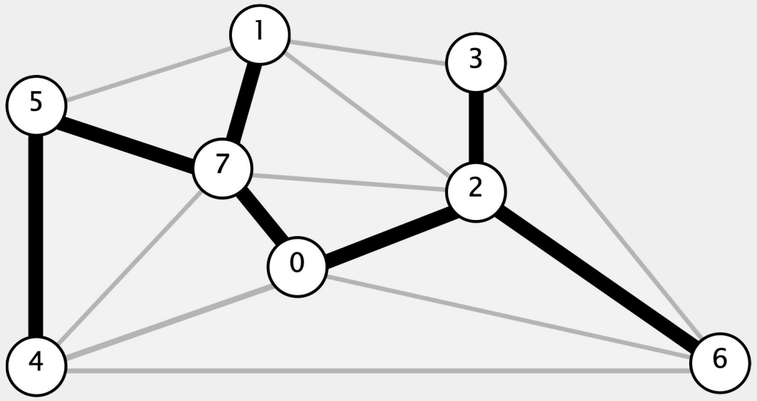
Universidade de Fortaleza

2. Algoritmo de Kruskal

2.1 Estratégia

Considere as arestas em ordem crescente de peso.

- *Adicione a próxima aresta à árvore T, a menos que essa adição crie um ciclo.*



MST 0-2 5-7 6-2 0-7 2-3 1-7 4-5

Arestas do grafo ordenadas por peso

0-7 0.16	4-5 0.35
2-3 0.17	1-2 0.36
1-7 0.19	4-7 0.37
0-2 0.26	0-4 0.38
5-7 0.28	6-2 0.40
1-3 0.29	3-6 0.52
1-5 0.32	6-0 0.58
2-7 0.34	6-4 0.93

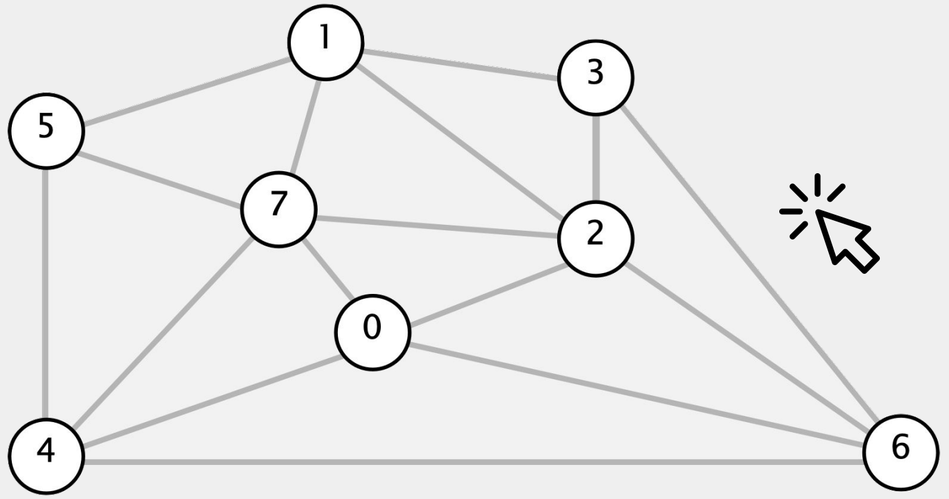
Figura 33 – Execução do algoritmo de Kruskal. As arestas são adicionadas em ordem crescente de peso, evitando ciclos. À direita, a lista das arestas do grafo ordenadas por peso. Fonte: Sedgewick, 2012.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

2. Algoritmo de Kruskal

2.1 Estratégia



Arestas do grafo ordenadas por peso

0-7 0.16	4-5 0.35
2-3 0.17	1-2 0.36
1-7 0.19	4-7 0.37
0-2 0.26	0-4 0.38
5-7 0.28	6-2 0.40
1-3 0.29	3-6 0.52
1-5 0.32	6-0 0.58
2-7 0.34	6-4 0.93

Figura 33 – Execução do algoritmo de Kruskal. Arestas são adicionadas em ordem crescente de peso, evitando ciclos. À direita, a lista das arestas do grafo ordenadas por peso. O processo continua até que todos os vértices estejam conectados, resultando em uma árvore com V−1 arestas. Fonte: Sedgewick, 2012.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

2. Algoritmo de Kruskal

2.2 Visualização

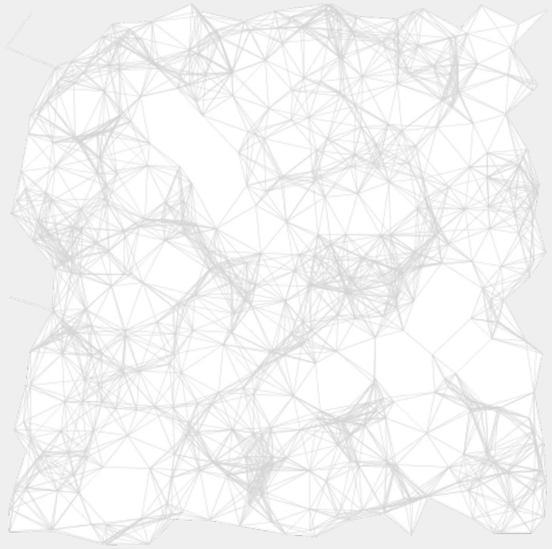


Figura 34 – Animação do algoritmo de Kruskal. As arestas são analisadas em ordem crescente de peso e adicionadas à MST se não formarem ciclos. O padrão seguido conecta inicialmente vértices mais próximos, expandindo componentes disjuntos até a conexão de todos os vértices. A verificação de ciclos é feita com o uso da estrutura de conjuntos disjuntos (Union-Find) para eficiência na união e busca dos componentes. Fonte: Sedgewick, 2012.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

Universidade de Fortaleza

2. Algoritmo de Kruskal

2.3 Prova de Corretude

Prop. *[Kruskal, 1956] O algoritmo de Kruskal computa a árvore geradora mínima (MST).*

Demonstração. *O algoritmo de Kruskal é um caso particular do algoritmo guloso para construção de MST.*

- *Suponha que o algoritmo de Kruskal colore a aresta e=v−w de preto.*
- *O corte considerado é o conjunto de vértices conectados a v na árvore parcial T.*
- *Nenhuma aresta de corte está colorida de preto.*
- *Nenhuma aresta de corte possui peso menor. Por quê?*

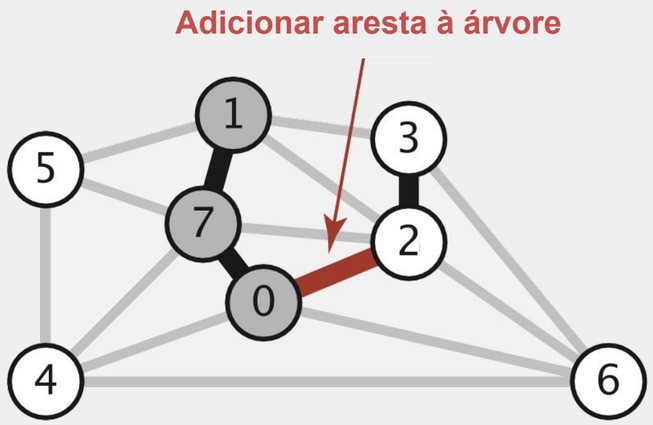


Figura 35 – Prova de corretude do algoritmo de Kruskal. A aresta destacada cruza o corte entre componentes disjuntos e possui o menor peso entre todas as arestas que cruzam esse corte, sendo segura para inclusão na árvore geradora mínima. Fonte: SEDGEWICK, 2017.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

2. Algoritmo de Kruskal

2.3 Desafio de implementação

***Desafio**: Adicionar a aresta v–w à árvore T criaria um ciclo? Se não, adicione-a.*

Qual a dificuldade?

- $E+V$
- V ← executar **DFS** a partir de v, verificar se w é alcançável (a árvore T tem no máximo V−1 arestas)
- $log\ V$
- $log^*\ V$ $^{(nota1)}$ ← use a estrutura de dados **union-find**!
- 1

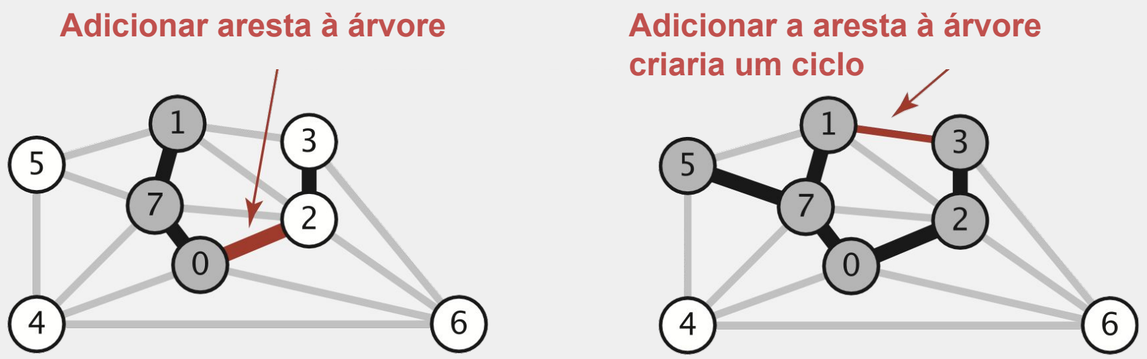


Figura 36 – Ilustração do desafio de implementação no algoritmo de Kruskal. Em (a), a aresta de corte v–w cruza une os componentes disjuntos e pode ser adicionada à árvore sem formar ciclo. Em (b), a mesma aresta conecta vértices já pertencentes ao mesmo componente, o que criaria um ciclo. A identificação dessa condição pode ser feita por DFS ou, de forma eficiente, com a estrutura de dados union-find. Fonte: SEDGEWICK, 2017.

Notas:
1 O logaritmo iterado de n (log* n) é o número de vezes que a função logaritmo (em base 2) deve ser aplicada recursivamente a um número n até que o resultado seja menor ou igual a 1.Na prática, log* n cresce tão lentamente que é considerado quase constante. Por exemplo, log* 65536 = 4 e log* 265536 = 5.

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

2. Algoritmo de Kruskal

2.3 Desafio de implementação

***Desafio**: Adicionar a aresta v–w à árvore T criaria um ciclo? Se não, adicione-a.*

- ***Solução eficiente.** Usar a estrutura de dados union-find.*
- *Mantenha um conjunto para cada componente conexo em T.*
- *Se v e w estiverem no mesmo conjunto, então adicionar v–w criaria um ciclo.*
- *Para adicionar v–w à árvore T, una os conjuntos que contêm v e w.*

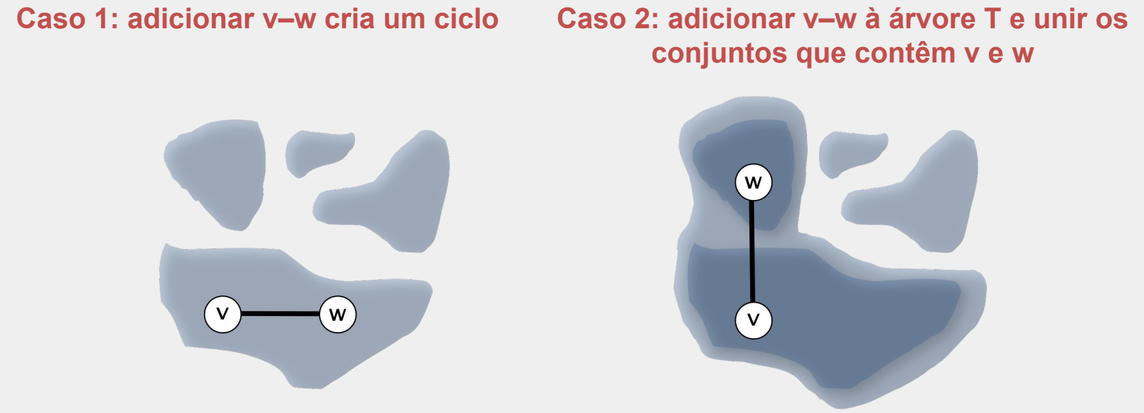


Figura 37 – Uso da estrutura de dados union-find no algoritmo de Kruskal. À esquerda, os vértices v e w pertencem ao mesmo conjunto, e a adição da aresta criaria um ciclo. À direita, v e w estão em conjuntos distintos, permitindo a adição segura da aresta à árvore T, seguida da união dos conjuntos. Fonte: SEDGEWICK, 2017. Fonte: SEDGEWICK, 2017.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

2. Algoritmo de Kruskal

2.3 Implementação em Java

```
public class KruskalMST
{
   private Queue<Edge> mst = new Queue<Edge>();

   public KruskalMST(EdgeWeightedGraph G)
   {
      MinPQ<Edge> pq = new MinPQ<Edge>();
      for (Edge e : G.edges())
         pq.insert(e);

      UF uf = new UF(G.V());
```

Declara uma fila encadeada (Queue) do tipo Edge para armazenar as arestas
Define o construtor da classe, que recebe um grafo ponderado como parâmetro (G) e constrói a MST.

MinPQ<Edge> é uma fila de prioridade mínima, onde as arestas são ordenadas pelo peso. Cada aresta e é inserida na fila de prioridade **pq**.

Cria uma estrutura de **Union-Find** com uma partição inicial para os V vértices.

Figura 37 – Uso da estrutura de dados union-find no algoritmo de Kruskal. À esquerda, os vértices v e w pertencem ao mesmo conjunto, e a adição da aresta criaria um ciclo. À direita, v e w estão em conjuntos distintos, permitindo a adição segura da aresta à árvore T, seguida da união dos conjuntos. Fonte: SEDGEWICK, 2017. Fonte: SEDGEWICK, 2017.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

Universidade de Fortaleza

2. Algoritmo de Kruskal

2.3 Implementação em Java

```
      while (!pq.isEmpty() && mst.size() < G.V()-1)
      {
         Edge e = pq.delMin();
         int v = e.either(), w = e.other(v);
         if (!uf.connected(v, w))
         {
            uf.union(v, w);
            mst.enqueue(e);
         }
      }
   }

   public Iterable<Edge> edges()
   {  return mst;  }
}
```

Enquanto a **PQ** não estiver vazia, e a **MST** ainda não tiver V−1 aresta

Remove a aresta de menor peso da fila de prioridades.
Obtém os dois vértices v e w conectados pela aresta e.

Testa se os vértices v e w já pertencem ao mesmo componente.

Une os conjuntos dos vértices v e w.

Adiciona a aresta e à fila **mst**.

Método que retorna um iterável com todas as arestas da **MST** construída.

Figura 37 – Uso da estrutura de dados union-find no algoritmo de Kruskal. À esquerda, os vértices v e w pertencem ao mesmo conjunto, e a adição da aresta criaria um ciclo. À direita, v e w estão em conjuntos distintos, permitindo a adição segura da aresta à árvore T, seguida da união dos conjuntos. Fonte: SEDGEWICK, 2017. Fonte: SEDGEWICK, 2017.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

Universidade de Fortaleza

2. Algoritmo de Kruskal

2.4 Tempo de execução

O algoritmo de Kruskal computa a MST em tempo proporcional a E logE no pior caso.

Operação	Freqüência	Tempo por operação
construir pq	1	E logE
delete-min	E	logE
union	V	$\log^* V^\dagger$
connected	E	$\log^* V^\dagger$

† limite amortizado usando união ponderada com compressão de caminho (weighted quick-union + path compression)

Observação. *Se as arestas já estiverem ordenadas, o tempo de execução passa a ter ordem de crescimento E* $\log^* V$ *(nota 1).*

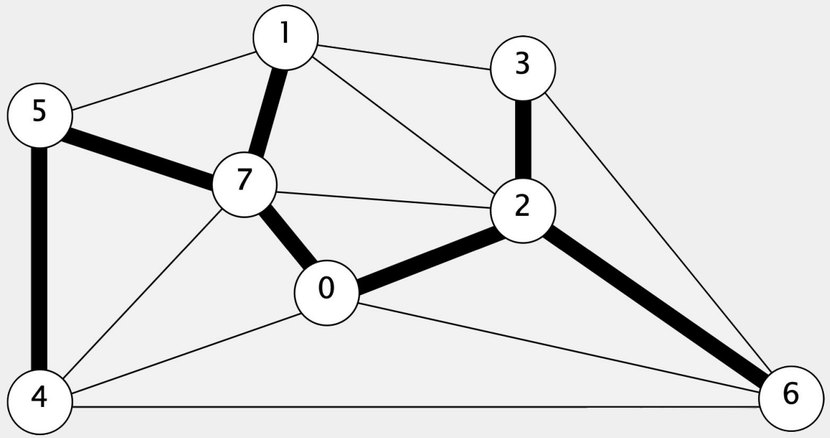
Notas:
1. Em qualquer aplicação prática, $\log^* V \leq 5$ neste contexto.

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

3. Algoritmo de Prim

3.1 Implementação ingênua

- *Comece com o vértice 0 e cresça a árvore T de forma gulosa.*
- *Adicione a T a aresta de menor peso com exatamente um dos extremos em T.*
- *Repita até ter V − 1 arestas.*



Arestas do grafo ordenadas por peso

0-7 0.16
2-3 0.17
1-7 0.19
0-2 0.26
5-7 0.28
1-3 0.29
1-5 0.32
2-7 0.34
4-5 0.35
1-2 0.36
4-7 0.37
0-4 0.38
6-2 0.40
3-6 0.52
6-0 0.58
6-4 0.93

MST 0-7 1-7 0-2 2-3 5-7 4-5 6-2

Figura 39 – Árvore geradora mínima (MST) obtida pelo algoritmo de Prim. A construção começa no vértice 0 e cresce a árvore T de forma gulosa, adicionando a cada passo a aresta de menor peso que conecta exatamente um vértice em T a um fora de T. Fonte: Sedgewick, 2012.

Notas:

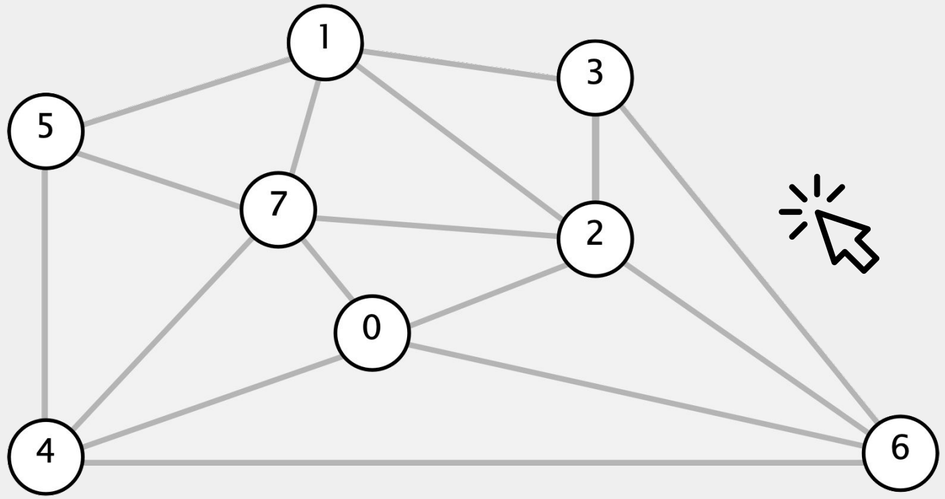
1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

Universidade de Fortaleza

3. Algoritmo de Prim

3.1 Implementação ingênua

- ***Objetivo:*** *Encontrar a aresta de menor peso com exatamente um dos extremos em T.*



Arestas do grafo ordenadas por peso

0-7 0.16	4-5 0.35
2-3 0.17	1-2 0.36
1-7 0.19	4-7 0.37
0-2 0.26	0-4 0.38
5-7 0.28	6-2 0.40
1-3 0.29	3-6 0.52
1-5 0.32	6-0 0.58
2-7 0.34	6-4 0.93

Figura 39 – Execução do algoritmo de Prim. A construção começa no vértice 0 e cresce a árvore T de forma gulosa, adicionando a cada passo a aresta de menor peso que conecta exatamente um vértice em T a um fora de T. O processo se repete até que a árvore contenha V − 1 arestas. Fonte: Sedgewick, 2012.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

Universidade de Fortaleza

3. Algoritmo de Prim

3.2 Implementação preguiçosa (lazy)

- ***Desafio**: Encontrar a aresta de menor peso com exatamente uma extremidade em T.*
- ***Solução**: Manter uma PQ de arestas com (pelo menos) uma extremidade em T.*
  - *Chave = aresta; prioridade = peso da aresta.*
  - *Usar delete-min para determinar a próxima aresta e = v–w a ser adicionada a T.*
  - *Desconsiderar se ambas as extremidades v e w estiverem marcadas (ambas pertencem a T)[1].*
  - *Caso contrário, seja w o vértice não marcado (fora de T):*
    - *adicionar à PQ qualquer aresta incidente a w (assumindo que a outra extremidade não está em T);*
    - *adicionar a aresta e a T e marcar w.*

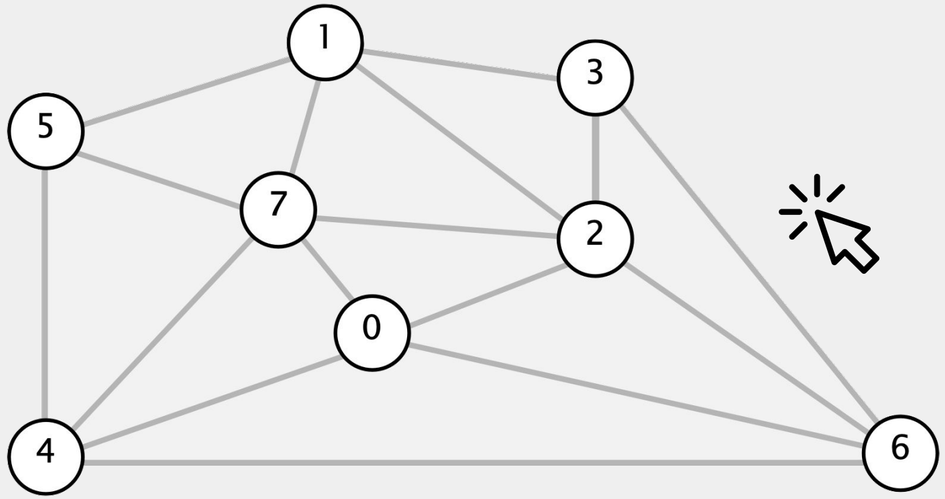
Notas:
1. Algumas arestas ficam obsoletas na fila. Por exemplo, se ambas as extremidades da aresta já estiverem na árvore, essa aresta não é mais útil.
2. Versão preguiçosa do algoritmo adiciona arestas a PQ sem pensar, descarta depois. Consome tempo e espaço extras, pois mantém arestas redundantes na fila.

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

3. Algoritmo de Prim

3.2 Implementação preguiçosa (lazzy)

- ***Objetivo:*** *Encontrar a aresta de menor peso com exatamente um dos extremos em T.*



Arestas do grafo ordenadas por peso

0-7 0.16	4-5 0.35
2-3 0.17	1-2 0.36
1-7 0.19	4-7 0.37
0-2 0.26	0-4 0.38
5-7 0.28	6-2 0.40
1-3 0.29	3-6 0.52
1-5 0.32	6-0 0.58
2-7 0.34	6-4 0.93

Figura 39 – Execução do algoritmo de Prim. A construção começa no vértice 0 e cresce a árvore T de forma gulosa, adicionando a cada passo a aresta de menor peso que conecta exatamente um vértice em T a um fora de T. O processo se repete até que a árvore contenha V − 1 arestas. Fonte: Sedgewick, 2012.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

Universidade de Fortaleza

3. Algoritmo de Prim

3.2 Implementação preguiçosa (lazzy)

```
public class LazyPrimMST
{
   private boolean[] marked;   // MST vertices
   private Queue<Edge> mst;    // MST edges
   private MinPQ<Edge> pq;     // PQ of edges

    public LazyPrimMST(WeightedGraph G)
    {
        pq = new MinPQ<Edge>();
        mst = new Queue<Edge>();
        marked = new boolean[G.V()];
        visit(G, 0);
```

Estruturas de Dados

EdgeWeightedGraph: representa um grafo ponderado com arestas não direcionadas.
Edge: representa uma aresta com peso, e dois vértices finais.
MinPQ<Edge>: fila de prioridade mínima para selecionar a aresta de menor peso com um extremo no MST.
Queue<Edge>: fila para armazenar as arestas que fazem parte da MST.
boolean[] marked: vetor booleano que indica se um vértice já pertence à MST.

visit(G, 0): Começa o algoritmo a partir do vértice 0, marcando-o e inserindo suas arestas válidas em pq (assume que G é conexo)

Figura 39 – Execução do algoritmo de Prim. A construção começa no vértice 0 e cresce a árvore T de forma gulosa, adicionando a cada passo a aresta de menor peso que conecta exatamente um vértice em T a um fora de T. O processo se repete até que a árvore contenha V − 1 arestas. Fonte: Sedgewick, 2012.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

3. Algoritmo de Prim

3.2 Implementação preguiçosa (lazzy)

```
        while (!pq.isEmpty() && mst.size() < G.V() - 1)
        {
           Edge e = pq.delMin();
           int v = e.either(), w = e.other(v);
           if (marked[v] && marked[w]) continue;
           mst.enqueue(e);
           if (!marked[v]) visit(G, v);
           if (!marked[w]) visit(G, w);
        }
    }
}
```

Remove repetidamente da fila de prioridade a aresta de menor peso e = v–w

Ignora se ambos os extremos estiverem em T (a árvore)

Adicione a aresta e à árvore

Adiciona v ou w à árvore

Figura 39 – Execução do algoritmo de Prim. A construção começa no vértice 0 e cresce a árvore T de forma gulosa, adicionando a cada passo a aresta de menor peso que conecta exatamente um vértice em T a um fora de T. O processo se repete até que a árvore contenha V − 1 arestas. Fonte: Sedgewick, 2012.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

Universidade de Fortaleza

3. Algoritmo de Prim

3.2 Implementação preguiçosa (lazzy)

```
private void visit(WeightedGraph G, int v)
{
   marked[v] = true;
   for (Edge e : G.adj(v))
      if (!marked[e.other(v)])
         pq.insert(e);
}

public Iterable<Edge> mst()
{  return mst;  }
```

Adiciona v a T.

Para cada aresta e = v–w, adicione à fila de prioridade se w ainda não estiver em T.

Figura 39 – Execução do algoritmo de Prim. A construção começa no vértice 0 e cresce a árvore T de forma gulosa, adicionando a cada passo a aresta de menor peso que conecta exatamente um vértice em T a um fora de T. O processo se repete até que a árvore contenha V − 1 arestas. Fonte: Sedgewick, 2012.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

Universidade de Fortaleza

3. Algoritmo de Prim

3.2 Complexidade

- ***Prop.**: O algoritmo de Prim (versão Lazy) calcula a MST em tempo proporcional a E log E e utiliza espaço extra proporcional a E (no pior caso).*
- ***Prova**:*

Operação	Frequência	Heap Binária
delete min	E	log E
insert	E	log E

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

3. Algoritmo de Prim

3.3 Implementação ansiosa (eager)

- ***Desafio**: Encontrar a aresta de menor peso com exatamente uma extremidade em T.*
- ***Solução**: Manter uma indexed PQ[1] de vértices conectados por uma aresta a T, onde a prioridade do vértice v é o menor peso da aresta que conecta v a T.*
  - *Remover o vértice de menor prioridade v e adicionar sua aresta associada e a T.*
  - *Atualizar a PQ considerando todas as arestas $e = v–x$ incidentes a v:*
    - *ignorar se x já está em T;*
    - *adicionar x à PQ se ainda não estiver nela;*
    - ***diminuir a prioridade** de x se v–x se tornar a menor aresta conectando x a T;*

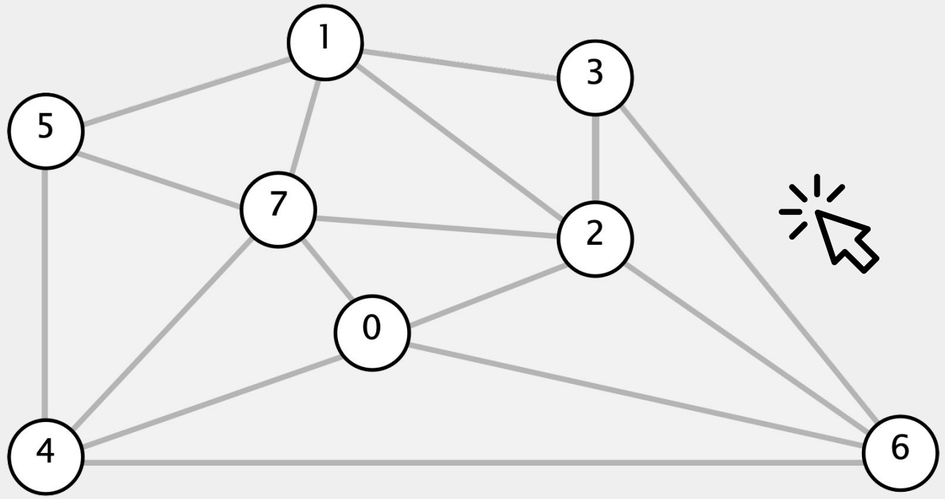
Notas:
1 A fila de prioridade (PQ) tem no máximo uma entrada por vértice.
2. Versão ansiosa do algoritmo atualiza a PQ assim que descobre arestas de melhor custo..

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

3. Algoritmo de Prim

3.3 Implementação ansiosa (eager)

- ***Objetivo:*** *Encontrar a aresta de menor peso com exatamente um dos extremos em T.*



Arestas do grafo ordenadas por peso

0-7 0.16	4-5 0.35
2-3 0.17	1-2 0.36
1-7 0.19	4-7 0.37
0-2 0.26	0-4 0.38
5-7 0.28	6-2 0.40
1-3 0.29	3-6 0.52
1-5 0.32	6-0 0.58
2-7 0.34	6-4 0.93

Figura 39 – Execução do algoritmo de Prim. A construção começa no vértice 0 e cresce a árvore T de forma gulosa, adicionando a cada passo a aresta de menor peso que conecta exatamente um vértice em T a um fora de T. O processo se repete até que a árvore contenha V − 1 arestas. Fonte: Sedgewick, 2012.

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

Universidade de Fortaleza

3. Algoritmo de Prim

3.3.1 Fila de Prioridade mínima (min-PQ) Indexada

public class IndexMinPQ<Key extends Comparable<Key>>	
IndexMinPQ(int N)	*create indexed priority queue with indices 0, 1, ..., N-1*
void insert(int i, Key key)	*associate key with index i*
void decreaseKey(int i, Key key)	*decrease the key associated with index i*
boolean contains(int i)	*is i an index on the priority queue?*
int delMin()	*remove a minimal key and return its associated index*
boolean isEmpty()	*is the priority queue empty?*
int size()	*number of entries in the priority queue*

Representa uma PQ indexada para chaves genéricas. Utiliza um heap binário juntamente com vetores para associar as chaves aos inteiros no intervalo entre 0 e N − 1.

Insere uma nova chave key na PQ associada ao índice i

Reduz a chave associada ao índice i para o valor especificado. Ou seja, atualiza o peso se encontrar conexão mais barata

Verifica se o índice i está presente na PQ, ou seja, se o item identificado por i já foi inserido e ainda não foi removido.

Remove e retorna o índice associado à menor chave da PQ.

Verifica se a PQ indexada está vazia.

Retorna o número de elementos atualmente presentes na PQ..

Figura 39 – Classe IndexMinPQ. A classe representa uma fila de prioridade mínima indexada para chaves genéricas. Ela suporta as operações usuais de: inserção (insert), remoção do mínimo (delMin) e redução da chave (decreaseKey) em tempo de execução Θ(log n) no pior caso.Fonte: Sedgewick, 2012

Notas:
1 A fila de prioridade (PQ) tem no máximo uma entrada por vértice.
2. Versão ansiosa do algoritmo atualiza a PQ assim que descobre arestas de melhor custo..

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

Universidade de Fortaleza

3. Algoritmo de Prim

3.3.1 Fila de Prioridade mínima (min-PQ) Indexada

- *Estende a lógica da minPQ tradicional, permitindo acesso direto pelo vértice v.*
- *Manter vetores paralelos, sendo k posição na heap:*
  - ***keys[v]**: qual é a prioridade do vértice v?*
  - ***qp[v]**: qual posição do heap está o vértice v?*
  - ***pq[k]**: qual vértice v está nesta posição k?*
- *A posição k pode mudar durante a reorganização do heap, enquanto v permanece.*
- *Atualizar a prioridade no pior caso, considerando PQ Heap Binária com v vértices:*
  - ***PQ comum**: localizar O(V) + reorganizar O(log V) = O(V).*
  - ***PQ Indexada**: localizar O(1) devido qp[] + reorganizar O(logV) = O(log V).*

Notas:

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

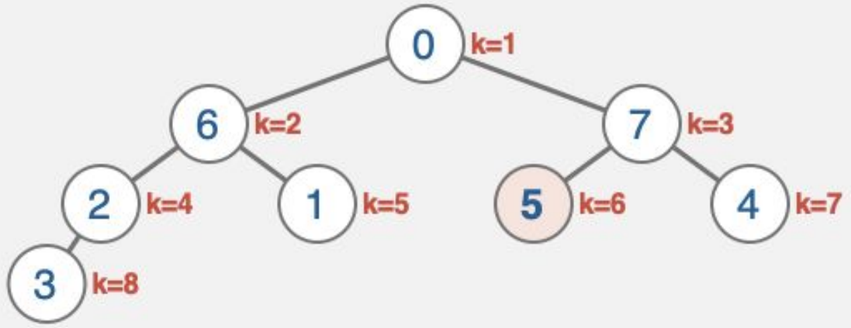
Universidade de Fortaleza

3. Algoritmo de Prim

3.3.1 Fila de Prioridade mínima (min-PQ) Indexada

Antes

índice	0	1	2	3	4	5	6	7	8
keys[v] = p	4	19	15	18	20	9	14	7	–
qp[v]	1	5	4	8	7	6	2	3	–
pq[k]	–	0	6	7	2	1	5	4	3



Depois

índice	0	1	2	3	4	5	6	7	8
keys[v] = p	4	19	15	18	20	3	14	7	–
qp[v]	3	5	4	8	7	1	2	6	–
pq[k]	–	5	6	0	2	1	7	4	3

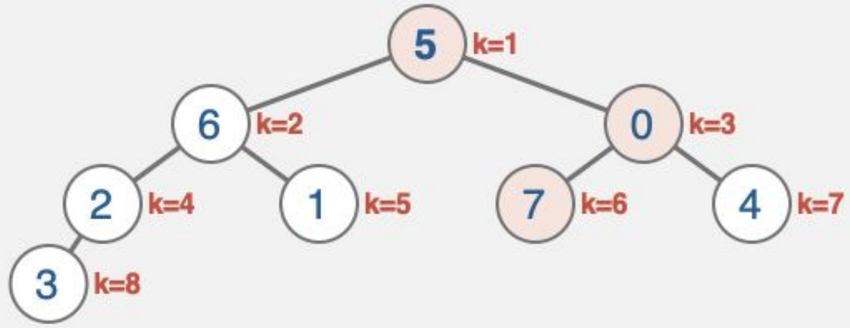


Figura 40 – Localização e atualização de prioridade em uma min-PQ indexada. O vetor qp[] permite localizar o vértice 5 na posição k = 6 do heap em O(1). Após a redução de sua prioridade de keys[5] = 9 para keys[5] = 3, o vértice sobe até a posição k = 1. Essa reorganização custa O(log V) no pior caso e atualiza os vetores pq[] e qp[]. Fonte: adaptação de Sedgewick, 2012.

Notas:
1. Após keys[5] = 3, o vértice 5 troca de posição com o 7, pois satisfazem keys[5] < keys[7] (3 < 7): pq[3] = 5, pq[6] = 7, qp[5] = 3 e qp[7] = 6. Depois, troca com o 0, pois keys[5] < keys[0] (3 < 4): pq[1] = 5, pq[3] = 0, qp[5] = 1 e qp[0] = 3. Em cada troca, pq[] registra quem ocupa cada posição, e qp[] registra a posição de cada vértice.

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

3. Algoritmo de Prim

3.4 Complexidade

- *Depende da implementação da fila de prioridade (PQ):*

Implementação PQ	insert	delete-min	descrease-key	total
Array	1	V	1	V^2
Heap binário	log V	log V	log V	E log V
Heap D-ário[1] (Johnson 1975)	$\log_d V$	d $\log_d V$	$\log_d V$	E $\log_{E/V} V$
Heap Fibonacci (Fredman-Tarjan 1984)	1*	log V*	1*	E + V log V

- ❖ *Implementação com array é ideal para grafos densos.*
- ❖ *Heap binário é muito mais rápido para grafos esparsos.*
- ❖ *Heap 4-ário vale o esforço em situações onde o desempenho é crítico.*
- ❖ *Heap de Fibonacci é o melhor em teoria, mas não compensa ser implementado na prática.*

Notas:
1 Um D-way heap (ou heap D-ário) é uma generalização do heap binário, onde cada nó interno tem até D filhos (em vez de apenas 2 como no binário)..

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

Obrigado

e até a próxima aula!